Module 05: Git and Version Control

Learning Objectives

By the end of this module, you will be able to:

- Explain why version control exists and what problem it solves
- Describe Git's internal architecture (working tree, staging area, repository)
- Use all essential Git commands confidently
- Create and manage branches
- Merge branches and resolve conflicts
- Push to and pull from remote repositories
- Follow a real-world Git workflow

5.1 Why Version Control?

Without version control:

```
project/
├── main.py
├── main_v2.py
├── main_v2_final.py
├── main_v2_final_ACTUAL.py
├── main_v2_final_ACTUAL_fixed.py
└── main_backup_dec15.py
```

With version control:

```
project/
└── main.py    ← One file. Full history. Every version recoverable.
```

Version control is not optional. It is the baseline professional practice. Every serious software project in history uses it.

5.2 Git's Architecture — The Mental Model

Git has three areas. Understanding these is the single most important thing about Git.

```
┌──────────────────┐    git add     ┌──────────────────┐   git commit
┌──────────────────┐
│  WORKING TREE    │ ──────────────► │  STAGING AREA    │ ──────────────► │
REPOSITORY     │
│  (your files)    │                │  (index)         │                 │
```

```
(.git/)        |
|                | ←──────────── |                |               |
|
|  Edit files    |  git restore  |  Preview what  |               |
Permanent      |
|  here          |               |  will be in    |               |
history        |
|                |               |  next commit   |               |
|
└────────────────┘               └────────────────┘
└────────────────┘
```

Working Tree: The files you see and edit. Your actual project directory.

Staging Area (Index): A preparation zone. You explicitly choose which changes go into the next commit. This is Git's killer feature — you can commit partial changes.

Repository (.git/): The permanent history. A database of all commits, stored in the hidden `.git` directory.

Analogy

Think of it as packing a box to ship:

1. **Working tree** = items scattered on your desk
2. **Staging area** = items you've placed in the box (but haven't sealed)
3. **Commit** = sealing the box and labeling it with a description

You don't have to put everything in the box. You choose what goes in each shipment.

5.3 Essential Commands

`git init` — Initialize a Repository

```
mkdir my-project
cd my-project
git init
```

This creates a `.git/` directory. Your project is now a Git repository.

```
ls -la .git/
# HEAD        ← Points to current branch
# config      ← Repository configuration
# objects/    ← Where Git stores data
# refs/       ← Branch and tag references
```

`git status` — Check State

The most-used command. Run it constantly.

```
git status
```

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
        modified:   app.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new_file.py

no changes added to commit (use "git add" to stage changes)
```

States a file can be in:

```
Untracked ──git add──► Staged ──git commit──► Committed
    │                    │                        │
    │                    │ git restore --staged   │
    │                    ◄────────────────────────┘
    │                                   git restore
    ◄──────────────────────────────────────────────┘
```

Exercise 5.1: The Git Lifecycle

```
# Create a new repo
mkdir git-practice && cd git-practice
git init

# Create a file
echo "Hello Git" > hello.txt
git status                    # Untracked

# Stage it
git add hello.txt
git status                    # Staged (Changes to be committed)

# Commit it
git commit -m "Add hello.txt"
git status                    # Clean (nothing to commit)

# Modify it
echo "More content" >> hello.txt
git status                    # Modified (Changes not staged)
```

```
# Stage and commit
git add hello.txt
git commit -m "Update hello.txt with more content"
```

`git add` — Stage Changes

```
git add file.txt              # Stage one file
git add file1.txt file2.txt   # Stage multiple files
git add .                     # Stage ALL changes in current directory
git add -p                    # Interactive: choose which hunks to stage
```

`git add .` is convenient but dangerous. It stages everything, including files you might not want (logs, secrets, build artifacts). Use `.gitignore` to prevent this.

`git commit` — Save a Snapshot

```
git commit -m "Add user authentication"
git commit                     # Opens editor for longer message
```

Commit message conventions:

```
# Good messages (imperative mood, explain WHY)
"Add user authentication with JWT"
"Fix null pointer in payment processing"
"Refactor database connection pooling"
"Remove deprecated API endpoints"

# Bad messages
"fix"
"updates"
"WIP"
"asdfasdf"
"fixed the thing"
```

A commit message should complete the sentence: "If applied, this commit will ___."

`git log` — View History

```
git log                        # Full log
git log --oneline              # Compact (one line per commit)
git log --oneline --graph      # With branch visualization
git log -5                     # Last 5 commits
git log --author="devak"       # By author
```

```
git log -- app.py              # History of specific file
git log --since="2025-01-01"   # Since a date
```

Reading a log entry:

```
commit a1b2c3d4e5f6... (HEAD -> main)
Author: Devak <devak@example.com>
Date:   Wed Jan 15 10:30:00 2025 -0500

    Add user authentication with JWT
```

- `a1b2c3d4...` = commit hash (unique ID)
- `HEAD -> main` = this is where you are, on branch "main"
- Author, date, and message follow

5.4 Branching

Branches let you work on features without affecting the main code.

Mental Model

```
main:      A ── B ── C ── D ── E
                     │
feature:             └── F ── G ── H
```

`main` continues its life. `feature` diverges at commit C and has its own commits.

Commands

```
git branch                     # List branches
git branch feature-login       # Create branch
git checkout feature-login     # Switch to branch
git checkout -b feature-login  # Create AND switch (shortcut)
git switch feature-login       # Modern alternative to checkout
git switch -c feature-login    # Create AND switch (modern)
git branch -d feature-login    # Delete branch (safe — warns if unmerged)
git branch -D feature-login    # Force delete branch
```

Exercise 5.2: Branching Practice

```
# Start on main
git checkout main
```

```
# Create and switch to a feature branch
git checkout -b feature-greeting

# Make changes
echo "def greet(name):" > greet.py
echo "    return f'Hello, {name}!'" >> greet.py
git add greet.py
git commit -m "Add greeting function"

# Add more to the feature
echo "" >> greet.py
echo "def farewell(name):" >> greet.py
echo "    return f'Goodbye, {name}!'" >> greet.py
git add greet.py
git commit -m "Add farewell function"

# Switch back to main
git checkout main
cat greet.py    # File doesn't exist on main!

# Switch back to feature
git checkout feature-greeting
cat greet.py    # It's here again
```

5.5 Merging

Merging brings changes from one branch into another.

Fast-Forward Merge

When main hasn't changed since the branch was created:

```
Before:
main:      A ── B ── C
                     │
feature:             └── D ── E

After merge:
main:      A ── B ── C ── D ── E
```

```
git checkout main
git merge feature-greeting
```

Three-Way Merge

When both branches have new commits:

```
Before:
main:      A ── B ── C ── F
                      │
feature:              └── D ── E

After merge:
main:      A ── B ── C ── F ── M  (M = merge commit)
                      │        │
feature:              └── D ── E┘
```

Merge Conflicts

Conflicts happen when both branches modify the same line.

```
# Git will tell you:
CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and then commit the result.
```

The conflict in the file looks like:

```
<<<<<<< HEAD
def greet():
    return "Hello from main!"
=======
def greet():
    return "Hello from feature!"
>>>>>>> feature-greeting
```

To resolve:

1. Open the file
2. Choose the correct version (or combine both)
3. Remove the `<<<<<<<`, `=======`, `>>>>>>>` markers
4. Stage and commit

```
# After editing the file to resolve:
git add app.py
git commit -m "Merge feature-greeting, resolve greeting conflict"
```

Exercise 5.3: Conflict Resolution

```
# Setup: create a conflict on purpose
git checkout main
```

```
echo "Main version" > conflict.txt
git add conflict.txt && git commit -m "Add conflict.txt on main"

git checkout -b conflict-branch
echo "Branch version" > conflict.txt
git add conflict.txt && git commit -m "Modify conflict.txt on branch"

git checkout main
echo "Updated main version" > conflict.txt
git add conflict.txt && git commit -m "Update conflict.txt on main"

# Now merge — this will conflict
git merge conflict-branch
# CONFLICT!

# Resolve it
nano conflict.txt    # Choose the right content
git add conflict.txt
git commit -m "Resolve merge conflict in conflict.txt"
```

5.6 Remote Repositories

Remote repositories are copies of your repo hosted elsewhere (GitHub, GitLab, etc.).

```
┌──────────────┐      git push      ┌──────────────────┐
│ LOCAL REPO   │ ──────────────────► │  REMOTE REPO     │
│ (your        │                    │  (GitHub)        │
│  machine)    │ ◄────────────────── │                  │
└──────────────┘      git pull      └──────────────────┘
```

Commands

```
# Connect to a remote
git remote add origin https://github.com/user/repo.git
git remote -v                          # List remotes

# Push your branch to remote
git push -u origin main                # First push (-u sets upstream)
git push                               # Subsequent pushes

# Pull changes from remote
git pull origin main                   # Fetch + merge
git fetch origin                       # Fetch only (no merge)

# Clone an existing repo
git clone https://github.com/user/repo.git
git clone https://github.com/user/repo.git my-folder  # Into specific
folder
```

`git pull` = `git fetch` + `git merge`

`fetch` downloads changes but doesn't apply them. `pull` downloads and merges in one step. Using `fetch` first gives you a chance to review before merging.

5.7 .gitignore

A `.gitignore` file tells Git which files to ignore.

```
# .gitignore

# Python
__pycache__/
*.pyc
*.pyo
.venv/
venv/
*.egg-info/

# Environment
.env
.env.local

# IDE
.vscode/
.idea/
*.swp

# OS
.DS_Store
Thumbs.db

# Build
dist/
build/
*.log

# Database
*.sqlite3
*.db
```

Create this file **before your first commit**. Files already tracked by Git are not affected by `.gitignore` — you must `git rm --cached` them first.

5.8 The Complete Git Workflow

Solo Developer Workflow

```
# 1. Create feature branch
git checkout -b feature-user-profile

# 2. Work on feature
#    (edit files, test, iterate)

# 3. Stage and commit (multiple small commits)
git add models/user.py
git commit -m "Add user profile model"

git add routes/profile.py
git commit -m "Add profile API endpoints"

git add tests/test_profile.py
git commit -m "Add tests for profile endpoints"

# 4. Switch to main and merge
git checkout main
git pull origin main          # Get latest changes
git merge feature-user-profile

# 5. Push
git push origin main

# 6. Delete feature branch
git branch -d feature-user-profile
```

Team Workflow (GitHub Flow)

```
# 1. Pull latest main
git checkout main
git pull origin main

# 2. Create feature branch
git checkout -b feature-user-profile

# 3. Work, commit (same as above)
# ...

# 4. Push feature branch
git push -u origin feature-user-profile

# 5. Open Pull Request on GitHub
# 6. Team reviews code
# 7. Merge PR on GitHub
# 8. Pull merged main locally

git checkout main
git pull origin main
git branch -d feature-user-profile
```

5.9 Git Mental Model Diagram

```
                          ┌──────────────────────────────────┐
                          │          REMOTE (GitHub)         │
                          │                                  │
                          │   main: A—B—C—D—E—F              │
                          │                                  │
                          └──────────────────────────────────┘
                                     │                 │
                                push ▲                 │ fetch/pull
                                     │                 ▼
┌────────────────┐   add   ┌──────────┐   commit   ┌────────────────┐
│ WORKING TREE   │ ──────► │ STAGING  │ ─────────► │  LOCAL REPO    │
│                │         │  AREA    │            │                │
│ Edit files     │◄────────│          │            │ main: A—B—C—D  │
│ here           │restore  │          │            │ feature: A—B—X │
└────────────────┘         └──────────┘            └────────────────┘

Commands that move data:
  Working → Staging:     git add
  Staging → Working:     git restore --staged (unstage)
  Staging → Repo:        git commit
  Repo → Working:        git restore (discard changes)
  Local → Remote:        git push
  Remote → Local:        git pull (or fetch + merge)
```

5.10 Useful Git Commands Reference

```
# Undo last commit (keep changes staged)
git reset --soft HEAD~1

# Undo last commit (keep changes unstaged)
git reset HEAD~1

# Discard all uncommitted changes (DANGEROUS)
git checkout -- .

# See what changed in a file
git diff app.py

# See staged changes
git diff --staged

# See changes between branches
git diff main..feature

# Stash changes temporarily
```

```
git stash
git stash list
git stash pop                  # Apply and remove latest stash
git stash apply                # Apply but keep in stash

# Blame — who changed each line
git blame app.py

# Show a specific commit
git show a1b2c3d

# Amend the last commit message
git commit --amend -m "Better message"

# Tag a release
git tag v1.0.0
git push origin v1.0.0
```

Checkpoint Quiz

1. What are the three areas in Git's architecture?
2. What is the difference between `git add` and `git commit`?
3. What does `git status` show you?
4. How do you create and switch to a new branch in one command?
5. What happens when you merge two branches that modified the same line?
6. What is the difference between `git pull` and `git fetch`?
7. Why should you create a `.gitignore` before your first commit?
8. What does `HEAD` refer to?
9. What command shows the commit history?
10. Why are small, focused commits better than large, mixed commits?

Common Mistakes

1. **Committing everything at once.** Make small, logical commits. One feature or fix per commit.
2. **Writing bad commit messages.** "fix" tells you nothing. "Fix null check in user validation" tells you everything.
3. **Not using `.gitignore` from the start.** Once a file is tracked, `.gitignore` won't help.
4. **Committing secrets.** `.env` files, API keys, passwords. Never. If you accidentally commit a secret, it's in the history forever (you need `git filter-branch` or BFG to clean it).
5. **Working directly on main.** Always branch. Even for small changes.
6. **Not pulling before pushing.** Always `git pull` before `git push` to avoid conflicts.
7. **Panicking at merge conflicts.** They are normal. Read the markers, choose the right code, remove the markers.
8. **Using `git add .` blindly.** Always `git status` first to see what you're staging.

Mini Project: Git Portfolio

1. Create a new repository called `git-portfolio`
2. Initialize it with a README.md
3. Create a `.gitignore` for Python projects
4. Create a branch called `feature-about`
5. Add an `about.md` file with your background
6. Commit it with a good message
7. Switch back to main
8. Create a different branch called `feature-projects`
9. Add a `projects.md` file listing 3 projects you want to build
10. Merge both branches into main (handle any conflicts)
11. Push to GitHub
12. Verify the history with `git log --oneline --graph --all`

Next Module

Proceed to `06_python_backend_basics.md` →